

QuickDelivery - Documentação

Aluno: Joaquim Vilela

Sprint: 1 - Arquitetura e Backend REST

1. Proposta do Domínio

1.1 Descrição

O **QuickDelivery** é uma plataforma de entregas sob demanda que conecta dois perfis distintos de usuário: o **cliente**, que solicita o transporte de um item entre dois endereços, e o **prestador de serviços** (entregador), que aceita a demanda e executa a entrega. O sistema implementa o ciclo de vida da solicitação: criação, aceite, execução, conclusão ou cancelamento.

1.2 Justificativa

- **Distinção clara entre cliente e prestador**, exigida no enunciado. Cliente e entregador possuem papéis, permissões e fluxos diferentes.
- **Comunicação naturalmente assíncrona**, alinhada ao princípio de EDA. Eventos como entrega criada, aceita e atualizada serão publicados no MOM na Sprint 2.
- **Domínio simples e verificável**, permitindo foco nos requisitos arquiteturais: REST, persistência, autenticação, autorização, Clean Architecture e, nas próximas sprints, RabbitMQ.
- **Escopo controlado**, com fluxo principal curto: criar -> aceitar -> em andamento -> entregue.

1.3 Perfis de Usuário

Cliente. Solicita entregas, lista suas próprias solicitações, acompanha status e pode cancelar uma entrega antes da execução.

Prestador de Serviços (entregador). Visualiza entregas pendentes, aceita demandas, acompanha entregas atribuídas a ele e atualiza o status da execução.

1.4 Principais Funcionalidades

Cliente

- Criar conta e autenticar.

- Solicitar entrega com origem, destino e descrição.
- Listar e consultar apenas suas próprias entregas.
- Cancelar entrega permitida pela máquina de estados.

Entregador

- Criar conta e autenticar.
- Listar entregas pendentes.
- Aceitar entrega usando seu próprio `deliverymanId`.
- Atualizar status para `IN_PROGRESS` e `DELIVERED`.

2. Backend REST

O backend expõe endpoints REST para autenticação, consulta de usuários por perfil e operação das entregas. O modelo de usuário foi unificado em `users`, com papel `CUSTOMER` ou `DELIVERYMAN`, substituindo a separação inicial entre clientes e prestadores. As entregas referenciam `customerId` e, após aceite, `deliverymanId`.

Endpoints principais:

Método	Rota	Descrição
POST	<code>/auth/signup</code>	Cria usuário cliente ou entregador.
POST	<code>/auth/login</code>	Retorna token de autenticação.
GET	<code>/auth/me</code>	Retorna o usuário autenticado.
GET	<code>/customers</code>	Lista clientes.
GET	<code>/deliverymen</code>	Lista entregadores.
POST	<code>/deliveries</code>	Cliente autenticado cria entrega própria.
GET	<code>/deliveries</code>	Lista entregas conforme perfil autenticado.
GET	<code>/deliveries/:id</code>	Consulta entrega com autorização por perfil.
PATCH	<code>/deliveries/:id/status</code>	Atualiza status com validação de transição.

As respostas usam códigos HTTP convencionais: `201` para criação, `200` para leitura/atualização, `400` para validação, `401` para autenticação ausente/inválida, `403` para ação proibida e `404` para entidades inexistentes. Erros seguem o formato `{ "error": "mensagem" }`.

3. Segurança e Autorização

O upgrade adiciona autenticação por token no padrão JWT assinado com HMAC SHA-256. O token contém `userId`, `role`, data de emissão e expiração. O middleware de autenticação interpreta o header `Authorization: Bearer <token>` e injeta o usuário autenticado na requisição.

Regras principais:

- Cliente só cria entregas para si mesmo.
- Cliente só lista, consulta e altera suas próprias entregas.
- Entregador vê entregas pendentes e entregas atribuídas a ele.
- Entregador só pode aceitar entrega usando seu próprio `deliverymanId`.
- Dados públicos de cliente/entregador não expõem senha.

4. Persistência

O schema Prisma usa PostgreSQL e contém:

- `users`: dados de autenticação, contato e papel do usuário.
- `deliveries`: solicitação de entrega, cliente, entregador opcional, endereços, descrição e status.
- `audit_logs`: registros simples de criação e atualização de entregas.

A migration da Sprint 1 foi recriada como uma migration inicial limpa, refletindo diretamente o modelo atual do projeto.

5. Organização do Código

O backend mantém separação em camadas:

- **routes**: mapeiam verbos e URLs para controllers.
- **controllers**: lidam com `req / res` e delegam a regra de negócio.
- **services**: concentram validações, autorização e máquina de estados.
- **repositories**: encapsulam acesso ao Prisma.
- **middlewares**: autenticação e tratamento centralizado de erros.
- **types**: tipos de usuário e status de entrega.

6. Validação via Postman

A coleção `postman/QuickDelivery.postman_collection.json` cobre o fluxo autenticado. Antes de executá-la, é necessário rodar:

```
npm run seed
```

Depois, a coleção realiza login do cliente e do entregador, armazena os tokens em variáveis, cria uma entrega e percorre o fluxo `PENDING -> ACCEPTED -> IN_PROGRESS -> DELIVERED` .